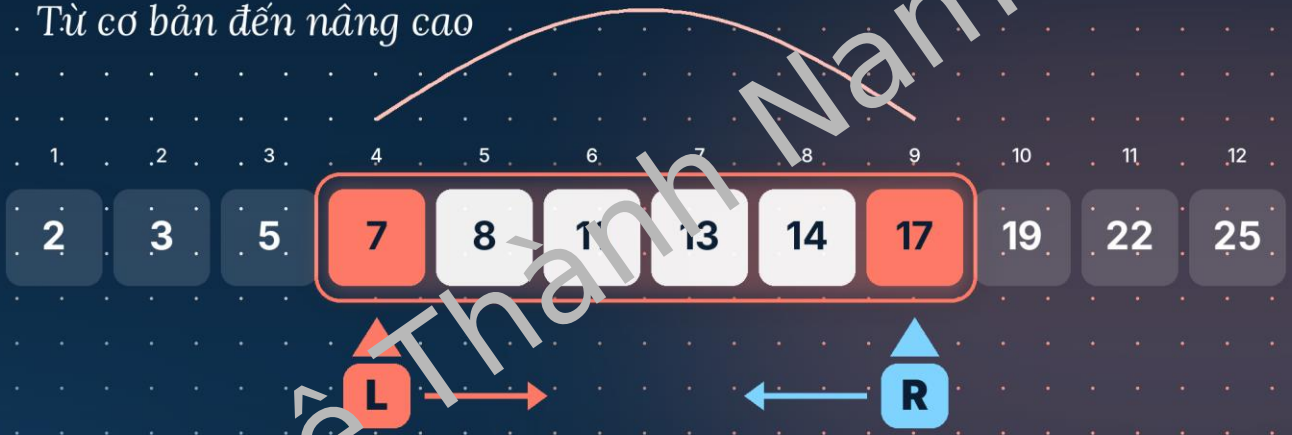
TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

KỸ THUẬT

HAI CON TRỎ

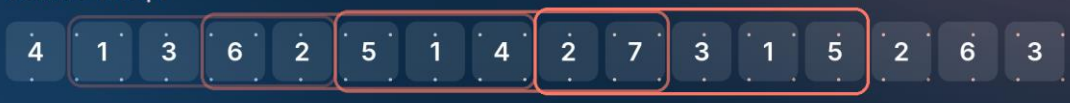
& cửa sổ trượt

Từ cơ bản đến nâng cao



Lê Thành Nam

CỬA SỔ TRƯỢT



TẬP 3 HƯỚNG DẪN GIẢI

NỘI DUNG

1. NỀN TẢNG HAI CON TRỎ
2. CỬA SỔ TRƯỢT
3. HAI CON TRỎ KẾT HỢP
4. HAI CON TRỎ NÂNG CAO

42 lời giải · Từ ngây thơ đến tối ưu · Bảng điểm thật

Tổng hợp và biên soạn: **Lê Thành Nam**

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho 42 bài tập của Tập 2. Mỗi lời giải không chỉ đưa ra chương trình đạt 100 điểm, mà đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ cách ngây thơ, đếm số phép tính để thấy bế tắc, rồi phá bế tắc bằng một quan sát về tính đơn điệu hay một cấu trúc dữ liệu phù hợp.

Nhiều bài có thêm một **lời giải trung gian** (đúng nhưng chưa đủ nhanh, ăn được một phần điểm) hoặc một **lời giải sai trông như đúng** (sai ở một trường hợp đặc biệt, một phép tràn số, một lần dùng cửa sổ trượt khi không có tính đơn điệu). Những lời giải ấy được giữ lại có chủ ý: Nhìn thấy một lời giải sai được bao nhiêu điểm và vì sao mất điểm là cách tốt nhất để không mắc lỗi đó trong phòng thi.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test chính thức với giới hạn thời gian và bộ nhớ ghi trong đề. Các con số trong **bảng điểm** cuối mỗi bài là điểm thực tế mà từng chương trình nhận được, không phải ước lượng.

Lê Thành Nam

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ của hai con trỏ, nhận ra mô hình quen thuộc ẩn sau câu chuyện.
2. Cách ngây thơ	Sơ đồ khối, chương trình và độ phức tạp của cách làm đầu tiên ai cũng nghĩ ra.
3. Bế tắc	Vì sao cách hiện tại không vượt qua được subtask lớn; đôi khi kèm một lời giải trung gian hoặc một lời giải sai "trông như đúng".
4. Phá bế tắc	Quan sát then chốt, tính đơn điệu, bất biến, hình vẽ chạy tay, lập luận vì sao đúng.
5. Cách tối ưu	Chương trình hoàn chỉnh và phân tích độ phức tạp theo tổng quãng đường của các con trỏ.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất.
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Lời khuyên: Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất mười lăm phút. Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh test để đối chiếu với cách ngây thơ của chính em.

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. NỀN TẢNG HAI CON TRỎ 6
Bài 1.1. CHỞ THUYỀN 6
Bài 1.2. CẶP GẦN NHẤT 11
Bài 1.3. NƯỚC MƯA 17
Bài 2.1. GIAO HAI TẬP 22
Bài 2.2. GHÉP CẶP 26
Bài 2.3. XÓA ĐOẠN 31
Bài 3.1. NÉN DÃY 36
Bài 3.2. TIÊU DIỆT 40
Bài 3.3. DÃY LẶP 44

CHƯƠNG 2. CỬA SỔ TRƯỢT 49
Bài 4.1. NHIỆT ĐỘ 49
Bài 4.2. HOÁN VỊ 53
Bài 4.3. TẦN SUẤT 49
Bài 5.1. TIẾT KIỆM 65
Bài 5.2. TÍCH NHỎ 67
Bài 5.3. TĂNG ĐỀU 74
Bài 6.1. KHÔNG LẶP 78
Bài 6.2. BỘ SƯU TẬP 82
Bài 6.3. TRÁI CÂY 87
Bài 7.1. SỐ LẺ 92
Bài 7.2. TỔNG ĐOẠN 96
Bài 7.3. MEX BẰNG K 94

CHƯƠNG 3. HAI CON TRỎ KẾT HỢP 105
Bài 8.1. TỔNG BA 105
Bài 8.2. BA CỬA HÀNG 109
Bài 8.3. PHÂN LOẠI TAM GIÁC 113
Bài 9.1. TỔNG NHỎ THỨ K 118
Bài 9.2. TÍCH NHỎ THỨ K 122
Bài 9.3. PHÂN SỐ THỨ K 126
Bài 10.1. VÒNG TRÒN 105
Bài 10.2. CẶP ĐIỂM 134
Bài 10.3. CHỨA TÂM 138
Bài 11.1. MUA SẮM 142
Bài 11.2. BA LÔ 40 147
Bài 11.3. CHIA HAI 152

Lê Thành Nam

CHƯƠNG 4. HAI CON TRỎ NÂNG CAO 157
Bài 12.1. BIÊN ĐỘ 157
Bài 12.2. ỔN ĐỊNH 41
Bài 12.3. TỔNG CÓ SỐ ÂM 165
Bài 13.1. PHÉP OR 170
Bài 13.2. ƯỚC CHUNG 128
Bài 13.3. BỘI CHUNG 179
Bài 14.1. TÌM TRONG MA TRẬN 185
Bài 14.2. VƯỜN RAU 189
Bài 14.3. ĐÚNG K CÂY 193

Lê Thành Nam

CHƯƠNG 1. NỀN TẢNG HAI CON TRỎ

Mỗi bài trong tập này được phân tích theo cùng một trình tự: hiểu bài toán qua hình vẽ, cách ngây thơ (kèm sơ đồ khối và số điểm đạt được), chỗ bế tắc, các bước phá bế tắc, chương trình tối ưu, các bẫy thường gặp và bảng điểm của từng cách làm. Bảng điểm được đo thật bằng bộ test chấm của sách.

Ở chương 1, ba câu hỏi quan trọng nhất là: Có được sắp xếp không? Phần tử nào chắc chắn vô dụng và có thể loại? Con trỏ nào di chuyển, theo chiều nào?

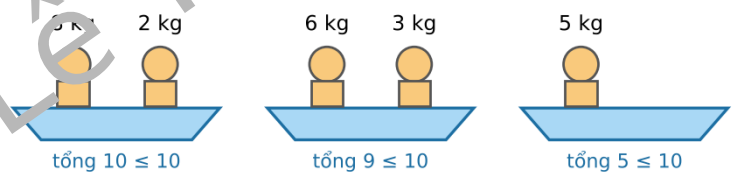
BÀI 1.1 CHỞ THUYỀN

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
THUYEN.INP	THUYEN.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1

1. Hiểu bài toán

Ta cần chia n bạn thành ít nhóm nhất, mỗi nhóm có một hoặc hai bạn với tổng cân nặng không quá C. Số thuyền bằng n trừ đi số **cặp** ghép được, nên bài toán tương đương: **Ghép được nhiều cặp nhất**, mỗi cặp có tổng ≤ C, mỗi bạn thuộc nhiều nhất một cặp.



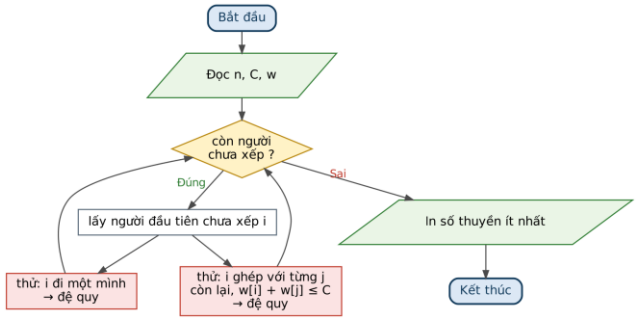
Hình 1.1.1. *Ví dụ 1: Ba thuyền cho năm bạn*

Thứ tự các bạn trong dữ liệu không quan trọng (ta chỉ quan tâm cân nặng), nên **được phép sắp xếp**.

2. Cách ngây thơ: Thử mọi cách xếp

Lấy bạn đầu tiên chưa được xếp. Bạn ấy hoặc đi một mình, hoặc đi cùng một bạn j nào đó (tổng ≤ C). Thử mọi khả năng bằng đệ quy và nhớ số thuyền ít nhất. Ta cắt nhánh khi số thuyền đã dùng không ít hơn kết quả tốt nhất hiện có.

Lê Thành Nam



Hình 1.1.2. *Sơ đồ khối cách thử mọi cách xếp*

Cách ngây thơ: Đệ quy thử mọi cách xếp

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int n;
long long c;
vector<long long> w;
vector<bool> dung;
int best;

void thu(int daXep, int soThuyen) {
    if (soThuyen >= best) return;
    int i = 0;
    while (i < n && dung[i]) i++;
    if (i == n) {
        best = soThuyen;
        return;
    }
    dung[i] = true;
    thu(daXep + 1, soThuyen + 1);
    for (int j = i + 1; j < n; j++)
        if (!dung[j] && w[i] + w[j] <= c) {
            dung[j] = true;
            thu(daXep + 2, soThuyen + 1);
            dung[j] = false;
        }
    dung[i] = false;
}

int main() {
    if (fopen("THUYEN.INP", "r")) {
        freopen("THUYEN.INP", "r", stdin);
        freopen("THUYEN.OUT", "w", stdout);
    }
    cin >> n >> c;
    w.resize(n);
    for (auto &x : w) cin >> x;
    dung.assign(n, false);
    best = n + 1;
    thu(0, 0);
    cout << best << '\n';
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: Số cách chia n người thành các nhóm một hoặc hai người là số "phép kết đôi" $T(n) = T(n-1) + (n-1)\cdot T(n-2)$, tăng nhanh hơn hàm mũ: $T(10) = 9496$, $T(20) \approx 2{,}4\cdot 10^{10}$. Chỉ chạy được với n khoảng 10 tới 12.

3. Bế tắc

- **Subtask 1** ($n \le 10$): Đệ quy chạy tức thì: **30 điểm**.
- **Subtask 2, 3**: Số cách xếp khổng lồ: 0 điểm.

Một cải tiến tự nhiên là **tham lam**: Sắp xếp tăng dần; xét bạn nặng nhất còn lại; ghép bạn ấy với bạn nhẹ nhất còn lại nếu được, không thì để bạn ấy đi một mình. Nếu cài đặt bằng cách duyệt mảng đánh dấu để tìm "bạn nhẹ nhất còn lại", mỗi lần tìm tốn O(n), tổng $O(n^2)$.

Lời giải trung gian: Tham lam, tìm bạn nhẹ nhất bằng cách duyệt, O(n^2^)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    if (fopen("THUYEN.INP", "r")) {
        freopen("THUYEN.INP", "r", stdin);
        freopen("THUYEN.OUT", "w", stdout);
    }
    int n;
    long long c;
    cin >> n >> c;
    vector<long long> w(n);
    for (auto &x : w) cin >> x;
    sort(w.begin(), w.end());
    vector<bool> dung(n, false);
    int kq = 0;
    for (int r = n - 1; r >= 0; r--) {
        if (dung[r]) continue;
        dung[r] = true;
        kq++;
        for (int l = 0; l < r; l++)
            if (!dung[l]) {
                if (w[l] + w[r] <= c) dung[l] = true;
                break;
            }
    }
    cout << kq << '\n';
    return 0;
}
```

Lê Thành Nam

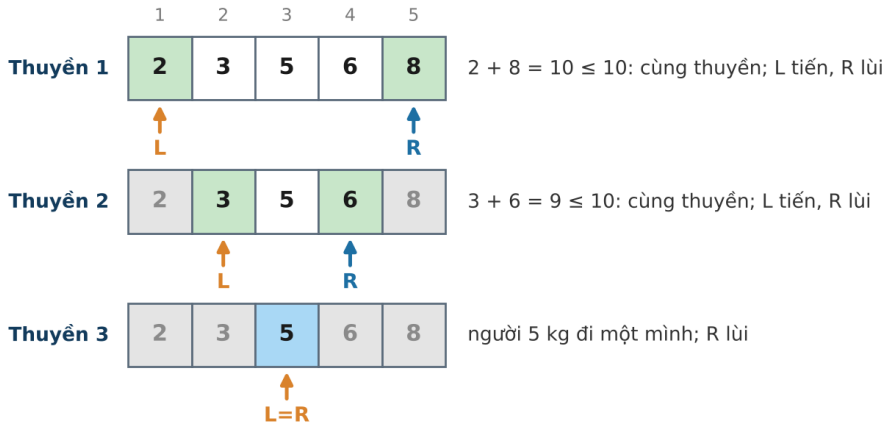
Lời giải này đúng (xem mục 4) và ăn thêm subtask 2 ($n \le 5000$): **60 điểm**. Với $n = 2 \cdot 10^5$ nó cần khoảng $2 \cdot 10^{10}$ bước.

> **ĐIỂM BẾ TẮC**
>
> Mỗi lần tìm "bạn nhẹ nhất chưa xếp" ta duyệt lại từ đầu mảng. Nhưng các bạn nhẹ được xếp theo đúng thứ tự từ nhẹ tới nặng: Vị trí "bạn nhẹ nhất còn lại" chỉ tiến về phía trước.

4. Phá bế tắc: Hai con trỏ ngược chiều

Sau khi sắp xếp $w_1 \le w_2 \le \ldots \le w_n$, đặt L = 1 (bạn nhẹ nhất còn lại) và R = n (bạn nặng nhất còn lại). Mỗi lượt, bạn R chắc chắn lên một thuyền mới:

- Nếu L < R và $w_L + w_R \le C$: Cho L đi cùng R, L tăng.
- R giảm, số thuyền tăng 1.



Hình 1.1.3. *Chạy tay với C = 10 trên dãy đã sắp xếp 2, 3, 5, 6, 8*

VÌ SAO THAM LAM ĐÚNG?

Xét bạn nặng nhất R.

- Nếu $w_1 + w_R > C$ thì R không ghép được với ai (bạn nhẹ nhất cũng không vừa): Trong mọi cách xếp, R đi một mình. Cho R đi một mình không làm mất gì.
- Nếu $w_1 + w_R \le C$: Lấy một cách xếp tối ưu bất kì. Nếu trong đó R đi cùng bạn $x \neq 1$, còn bạn 1 đi cùng bạn y, ta đổi chỗ x và 1. Thuyền của R giờ có R và 1, tổng $w_R + w_1 \le C$. Thuyền kia có x và y. Vì R nặng nhất nên $w_y \le w_R$, và vì x từng đi cùng R nên $w_x + w_R \le C$; suy ra $w_1 + w_y \le w_x + w_R \le C$. Số thuyền không đổi. Nếu bạn 1 đi một mình thì sau khi đổi x đi một mình: Vẫn hợp lệ. Nếu R đi một mình, chuyển bạn 1 sang thuyền của R không làm tăng số thuyền.
- Vậy luôn tồn tại cách tối ưu trong đó R đi cùng bạn nhẹ nhất (nếu vừa). Loại hai bạn này và lặp lại trên phần còn lại.

Đếm số phép tính: Mỗi lượt R giảm 1, nên đúng n lượt; L tăng không quá n lần. Cùng với sắp xếp: **O(n log n)**.

Lê Thanh Nam

5. Cách tối ưu

THUYEN.CPP: Sắp xếp và hai con trỏ, O(n log n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("THUYEN.INP", "r")) {
        freopen("THUYEN.INP", "r", stdin);
        freopen("THUYEN.OUT", "w", stdout);
    }
    int n;
    long long c;
    cin >> n >> c;
    vector<long long> w(n);
    for (auto &x : w) cin >> x;
    sort(w.begin(), w.end());
    int l = 0, r = n - 1, kq = 0;
    while (l <= r) {
        if (l < r && w[l] + w[r] <= c) l++;
        r--;
        kq++;
    }
    cout << kq << '\n';
    return 0;
}
```

Điều kiện vòng lặp là `l <= r` (không phải `l < r`): Khi L = R còn đúng một bạn chưa xếp, bạn ấy vẫn cần một thuyền. Còn điều kiện ghép đôi là `l < r` để không ghép một bạn với chính mình.

Lê Thành Nam

6. Bẫy thường gặp

- **Ghép hai bạn nhẹ nhất với nhau:** Tham lam sai hướng. Ví dụ C = 10, cân nặng 1, 2, 8, 9: Ghép (1, 2) trước thì 8 và 9 phải đi riêng, tổng 3 thuyền; đáp án đúng là 2 thuyền (9 cùng 1, 8 cùng 2). Hãy luôn bám theo chứng minh ở mục 4: Bạn **nặng nhất** quyết định.
- **Dùng `l < r` làm điều kiện vòng lặp:** Bỏ sót bạn cuối cùng khi số người lẻ.
- **Quên sắp xếp.**

7. Bảng điểm theo thuật toán

Thuật toán	Subtask 1 (30)	Subtask 2 (30)	Subtask 3 (40)	Tổng
Đệ quy thử mọi cách xếp	30	0	0	**30**
Tham lam, tìm kiếm tuyến tính $O(n^2)$	30	30	0	**60**
Sắp xếp + hai con trỏ O(n log n)	30	30	40	**100**